Compiler Lab 6 (BCSE307P)

Submitted by- **Sakshi Biyani**

Registration no- **22BLC1385**

L15+L16

To check whether a string is derivable or not from a given grammer

Aim: To implement a **bottom-up parsing** algorithm using a **queue-based approach** to simulate the parsing of a string based on a given **context-free grammar** (CFG). The program also provides **tracing** of the steps taken during the parsing process, allowing the user to see how the input string is being matched against the grammar productions.

Compiler used: GCC (GNU Compiler Collection)

Editor used : gedit

Algorithm

1. **Input and Grammar Setup:**

- The program first takes the number of **non-terminals** in the grammar.
- For each non-terminal:
  - The program takes the **name** of the non-terminal.
  - The number of **productions** for that non-terminal is inputted.
  - Each production is inputted as a string of symbols, where the symbols could be terminals or non-terminals.
- Then, the program takes the input string that needs to be parsed.

2. **Tokenization of Input:**

- The input string is split into **tokens** (characters) that will be matched against the grammar symbols.

3. **Queue Initialization:**

- The parsing process is simulated using a **queue**:
  - The **initial state** is created with the start symbol (first non-terminal) and an input position pointing to the beginning of the input string.
  - This state is enqueued to begin the parsing process.

4. **Parsing Process (Main Loop):**

- The main loop processes the queue:
  - **Dequeue a state** from the front of the queue.
  - Check if all symbols have been processed:

        - If the symbols are empty and the input position has reached the end of the input, the string is accepted.
    - If the front symbol is a **non-terminal**:
        - For each production of the non-terminal, **apply the production** by replacing the non-terminal with the symbols of the production.
        - The new state (with updated symbols) is enqueued.
    - If the front symbol is a **terminal**:
        - **Match it** with the current symbol from the input string.
        - If it matches, update the input position and move forward to the next state.
    - If a terminal or non-terminal can't match, the string is rejected.

5. **Tracing the Parsing Process:**

- For each step of the parsing process (both production applications and terminal matches):
    - Print the **current state**: including the symbols being processed and the current input position.
    - Print the **production applied** or the **terminal matched** for each step.

6. **Termination:**

- If the parsing finishes with the input string fully processed and accepted, print "String is accepted."
- Otherwise, print "String is rejected."

Code

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>


struct Production {

   char **symbols;

   int length;

};


struct NonTerminal {

   char *name;

   struct Production *productions;

   int num_productions;

};
```

```
struct State {
    char **symbols;
    int symbols_len;
    int input_pos;
};

struct QueueNode {
    struct State state;
    struct QueueNode *next;
};

char **split_into_tokens(const char *line, int *num_tokens) {
    int len = strlen(line);
    char **tokens = malloc(len * sizeof(char *));
    int count = 0;
    for (int i = 0; i < len; i++) {
        char c = line[i];
        if (c == '\0' || c == '\n') break;
        if (isspace(c)) continue;
        tokens[count] = malloc(2 * sizeof(char));
        tokens[count][0] = c;
        tokens[count][1] = '\0';
        count++;
    }
    *num_tokens = count;
    return tokens;
}
```

```
void print_trace(struct State state, char *current_symbol, int input_pos, int num_input_tokens) {
    printf("Tracing: ");
    printf("[Symbols: ");
    for (int i = 0; i < state.symbols_len; i++) {
        printf("%s ", state.symbols[i]);
    }
    printf(" | Input position: %d/%d] Current symbol: %s\n", input_pos, num_input_tokens, current_symbol);
}

int main() {
    int num_non_terminals;
    printf("Enter number of non-terminals: ");
    scanf("%d", &num_non_terminals);
    getchar();

    struct NonTerminal *non_terminals = malloc(num_non_terminals * sizeof(struct NonTerminal));

    for (int i = 0; i < num_non_terminals; i++) {
        char name[256];
        printf("Enter name of non-terminal %d: ", i + 1);
        fgets(name, sizeof(name), stdin);
        name[strcspn(name, "\n")] = '\0';
        non_terminals[i].name = strdup(name);

        printf("Enter number of productions for %s: ", name);
        int num_prods;
        scanf("%d", &num_prods);
        getchar();
        non_terminals[i].num_productions = num_prods;
```

```
    non_terminals[i].productions = malloc(num_prods * sizeof(struct Production));

    for (int j = 0; j < num_prods; j++) {
        char prod_str[256];
        printf("Enter production %d for %s as a string: ", j + 1, name);
        fgets(prod_str, sizeof(prod_str), stdin);
        prod_str[strcspn(prod_str, "\n")] = '\0';

        int prod_length;
        non_terminals[i].productions[j].symbols = split_into_tokens(prod_str, &prod_length);
        non_terminals[i].productions[j].length = prod_length;
    }
}

char input_str[256];
printf("Enter input string to parse: ");
fgets(input_str, sizeof(input_str), stdin);
input_str[strcspn(input_str, "\n")] = '\0';

int num_input_tokens;
char **input_tokens = split_into_tokens(input_str, &num_input_tokens);

struct QueueNode *front = NULL, *rear = NULL;
char **initial_symbols = malloc(sizeof(char *));
initial_symbols[0] = non_terminals[0].name;
struct State initial_state = { initial_symbols, 1, 0 };
struct QueueNode *initial_node = malloc(sizeof(struct QueueNode));
initial_node->state = initial_state;
initial_node->next = NULL;
```

```
front = rear = initial_node;


int accepted = 0;


printf("Tracing begins...\n");  // Start of tracing
while (front != NULL && !accepted) {
   struct QueueNode *current = front;
   front = front->next;
   if (front == NULL) rear = NULL;
   struct State state = current->state;
   if (state.symbols_len == 0) {
      if (state.input_pos == num_input_tokens) {
         accepted = 1;
      }
   } else {
      char *current_symbol = state.symbols[0];
      int is_nt = 0;
      struct NonTerminal *nt = NULL;
      for (int k = 0; k < num_non_terminals; k++) {
         if (strcmp(non_terminals[k].name, current_symbol) == 0) {
            is_nt = 1;
            nt = &non_terminals[k];
            break;
         }
      }
      print_trace(state, current_symbol, state.input_pos, num_input_tokens);  // Tracing here


      if (is_nt) {
         for (int p = 0; p < nt->num_productions; p++) {
```

```
            struct Production *prod = &nt->productions[p];
            int new_len = prod->length + state.symbols_len - 1;
            char **new_symbols = malloc(new_len * sizeof(char *));
            for (int i = 0; i < prod->length; i++) {
                new_symbols[i] = prod->symbols[i];
            }
            for (int i = 0; i < state.symbols_len - 1; i++) {
                new_symbols[prod->length + i] = state.symbols[i + 1];
            }
            struct State new_state = { new_symbols, new_len, state.input_pos };
            struct QueueNode *new_node = malloc(sizeof(struct QueueNode));
            new_node->state = new_state;
            new_node->next = NULL;
            if (rear) {
                rear->next = new_node;
                rear = new_node;
            } else {
                front = rear = new_node;
            }
            printf("  Applying production: %s -> ", nt->name);
            for (int i = 0; i < prod->length; i++) {
                printf("%s ", prod->symbols[i]);
            }
            printf("\n");
        }
    } else {
        if (state.input_pos < num_input_tokens && strcmp(current_symbol, input_tokens[state.input_pos]) == 0) {
            int new_pos = state.input_pos + 1;
            int new_len = state.symbols_len - 1;
```

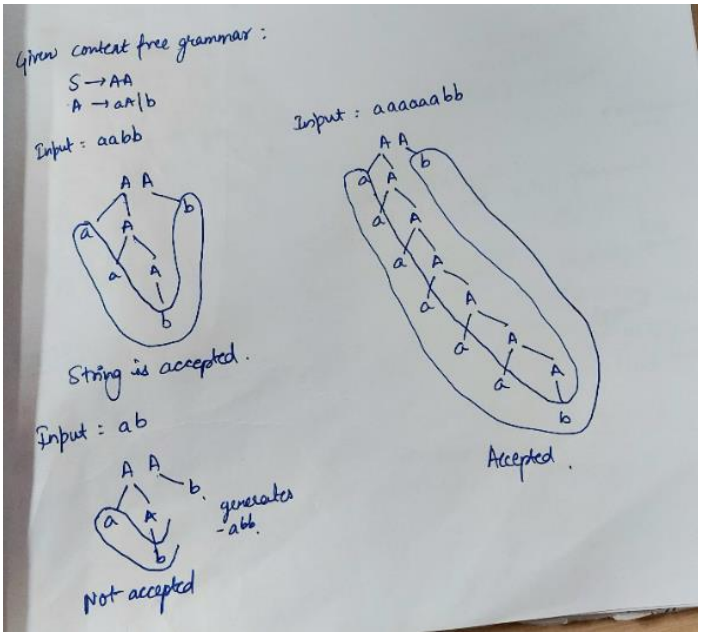
```
            char **new_symbols = NULL;
            if (new_len > 0) {
                new_symbols = malloc(new_len * sizeof(char *));
                for (int i = 0; i < new_len; i++) {
                    new_symbols[i] = state.symbols[i + 1];
                }
            }
            struct State new_state = { new_symbols, new_len, new_pos };
            struct QueueNode *new_node = malloc(sizeof(struct QueueNode));
            new_node->state = new_state;
            new_node->next = NULL;
            if (rear) {
                rear->next = new_node;
                rear = new_node;
            } else {
                front = rear = new_node;
            }
            printf("  Matching terminal: %s\n", current_symbol);
        }
    }
  }
  free(state.symbols);
  free(current);
}

for (int i = 0; i < num_non_terminals; i++) {
  for (int j = 0; j < non_terminals[i].num_productions; j++) {
    for (int k = 0; k < non_terminals[i].productions[j].length; k++) {
      free(non_terminals[i].productions[j].symbols[k]);
```

```
        }
        free(non_terminals[i].productions[j].symbols);
      }
      free(non_terminals[i].productions);
      free(non_terminals[i].name);
    }
    free(non_terminals);
    for (int i = 0; i < num_input_tokens; i++) {
      free(input_tokens[i]);
    }
    free(input_tokens);
    printf(accepted ? "String is accepted.\n" : "String is rejected.\n");
    return 0;
}
```

The given context free grammar is :

S-> AA

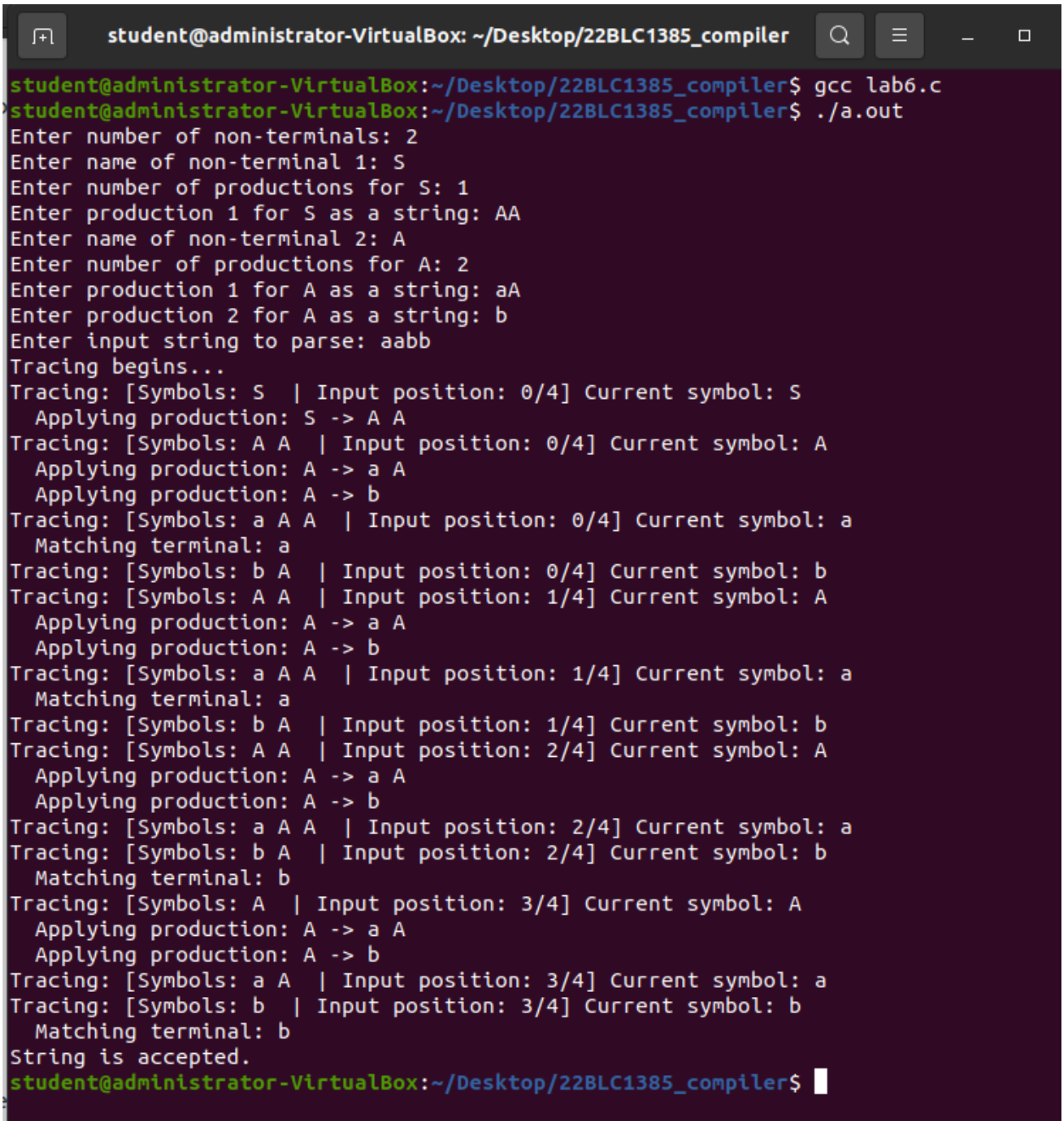
A->aA|b



Output

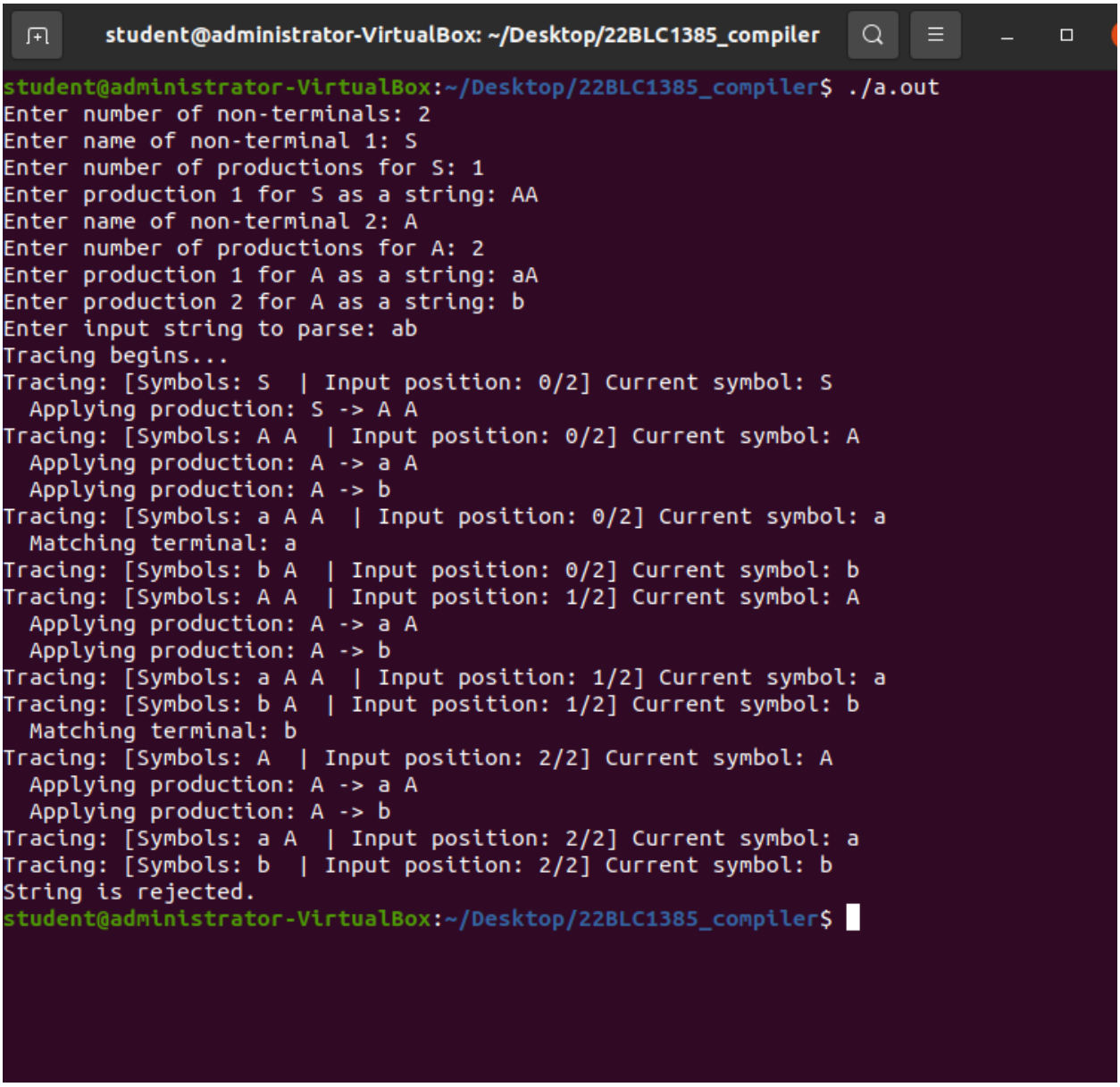
Case 1 :

Input : aabb



Case 2:

Input : ab



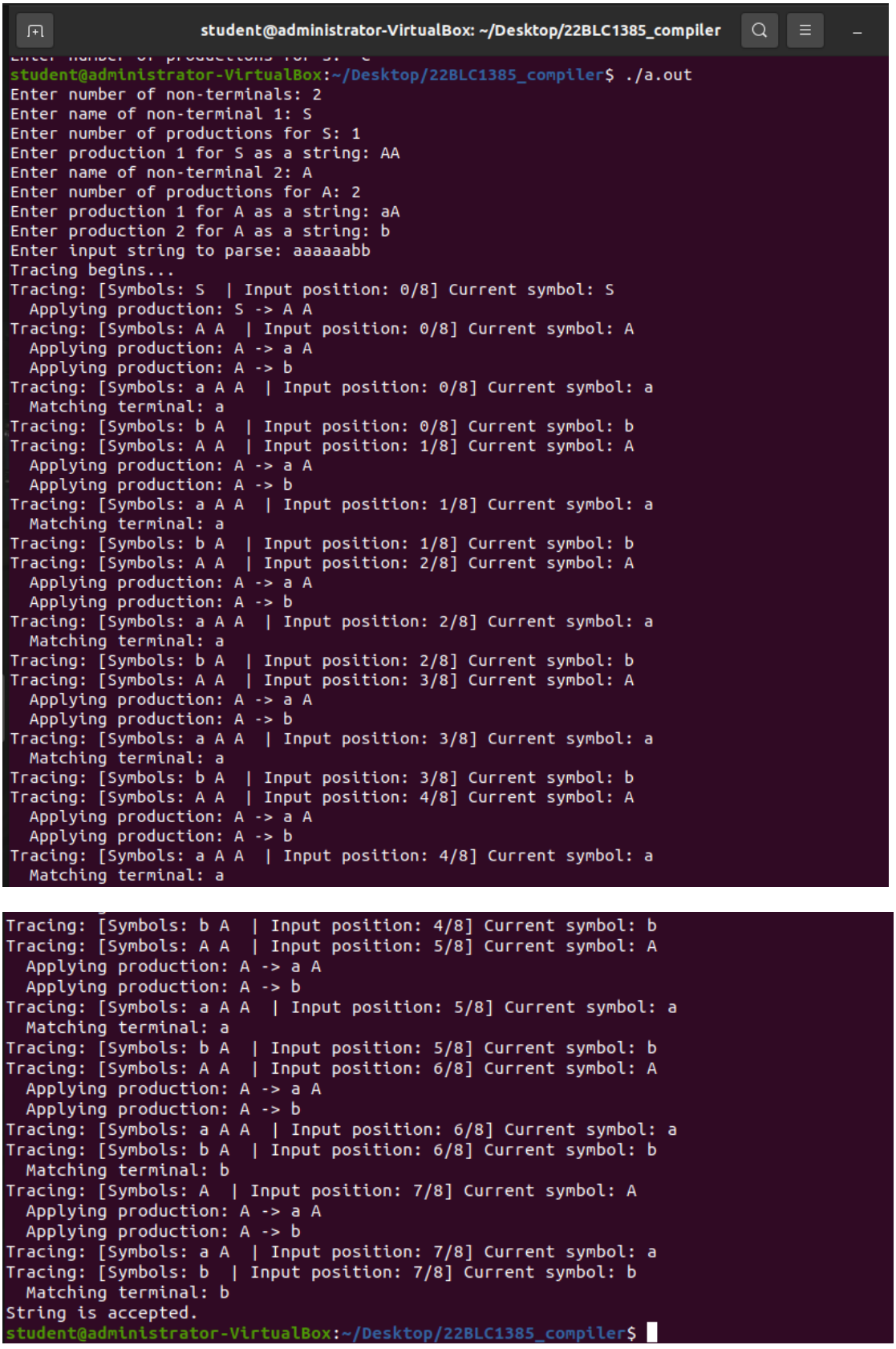
Case 3:

Input : ababab

```
student@administrator-VirtualBox:~/Desktop/22BLC1385_compiler$ ./a.out
Enter number of non-terminals: 2
Enter name of non-terminal 1: S
Enter number of productions for S: 1
Enter production 1 for S as a string: AA
Enter name of non-terminal 2: A
Enter number of productions for A: 2
Enter production 1 for A as a string: aA
Enter production 2 for A as a string: b
Enter input string to parse: ababab
Tracing begins...
Tracing: [Symbols: S  | Input position: 0/6] Current symbol: S
  Applying production: S -> A A
Tracing: [Symbols: A A  | Input position: 0/6] Current symbol: A
  Applying production: A -> a A
  Applying production: A -> b
Tracing: [Symbols: a A A  | Input position: 0/6] Current symbol: a
  Matching terminal: a
Tracing: [Symbols: b A  | Input position: 0/6] Current symbol: b
Tracing: [Symbols: A A  | Input position: 1/6] Current symbol: A
  Applying production: A -> a A
  Applying production: A -> b
Tracing: [Symbols: a A A  | Input position: 1/6] Current symbol: a
Tracing: [Symbols: b A  | Input position: 1/6] Current symbol: b
  Matching terminal: b
Tracing: [Symbols: A  | Input position: 2/6] Current symbol: A
  Applying production: A -> a A
  Applying production: A -> b
Tracing: [Symbols: a A  | Input position: 2/6] Current symbol: a
  Matching terminal: a
Tracing: [Symbols: b  | Input position: 2/6] Current symbol: b
Tracing: [Symbols: A  | Input position: 3/6] Current symbol: A
  Applying production: A -> a A
  Applying production: A -> b
Tracing: [Symbols: a A  | Input position: 3/6] Current symbol: a
Tracing: [Symbols: b  | Input position: 3/6] Current symbol: b
  Matching terminal: b
String is rejected.
student@administrator-VirtualBox:~/Desktop/22BLC1385_compiler$
```

Case 4

Input : aaaaaab


student@administrator-VirtualBox: ~/Desktop/22BLC1385_compiler
student@administrator-VirtualBox:~/Desktop/22BLC1385_compiler$ ./a.out
Enter number of non-terminals: 2
Enter name of non-terminal 1: S
Enter number of productions for S: 1
Enter production 1 for S as a string: AA
Enter name of non-terminal 2: A
Enter number of productions for A: 2
Enter production 1 for A as a string: aA
Enter production 2 for A as a string: b
Enter input string to parse: aaaaaabb
Tracing begins...
Tracing: [Symbols: S | Input position: 0/8] Current symbol: S
Applying production: S -> A A
Tracing: [Symbols: A A | Input position: 0/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A A | Input position: 0/8] Current symbol: a
Matching terminal: a
Tracing: [Symbols: b A | Input position: 0/8] Current symbol: b
Tracing: [Symbols: A A | Input position: 1/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A A | Input position: 1/8] Current symbol: a
Matching terminal: a
Tracing: [Symbols: b A | Input position: 1/8] Current symbol: b
Tracing: [Symbols: A A | Input position: 2/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A A | Input position: 2/8] Current symbol: a
Matching terminal: a
Tracing: [Symbols: b A | Input position: 2/8] Current symbol: b
Tracing: [Symbols: A A | Input position: 3/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A A | Input position: 3/8] Current symbol: a
Matching terminal: a
Tracing: [Symbols: b A | Input position: 3/8] Current symbol: b
Tracing: [Symbols: A A | Input position: 4/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A A | Input position: 4/8] Current symbol: a
Matching terminal: a
Tracing: [Symbols: b A | Input position: 4/8] Current symbol: b
Tracing: [Symbols: A A | Input position: 5/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A A | Input position: 5/8] Current symbol: a
Matching terminal: a
Tracing: [Symbols: b A | Input position: 5/8] Current symbol: b
Tracing: [Symbols: A A | Input position: 6/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A A | Input position: 6/8] Current symbol: a
Tracing: [Symbols: b A | Input position: 6/8] Current symbol: b
Matching terminal: b
Tracing: [Symbols: A | Input position: 7/8] Current symbol: A
Applying production: A -> a A
Applying production: A -> b
Tracing: [Symbols: a A | Input position: 7/8] Current symbol: a
Tracing: [Symbols: b | Input position: 7/8] Current symbol: b
Matching terminal: b
String is accepted.
student@administrator-VirtualBox:~/Desktop/22BLC1385_compiler$

Case 5:

Input : abb

```
student@administrator-VirtualBox:~/Desktop/22BLC1385_compiler$ ./a.out
Enter number of non-terminals: 2
Enter name of non-terminal 1: S
Enter number of productions for S: 1
Enter production 1 for S as a string: AA
Enter name of non-terminal 2: A
Enter number of productions for A: 2
Enter production 1 for A as a string: aA
Enter production 2 for A as a string: b
Enter input string to parse: abb
Tracing begins...
Tracing: [Symbols: S  | Input position: 0/3] Current symbol: S
  Applying production: S -> A A
Tracing: [Symbols: A A  | Input position: 0/3] Current symbol: A
  Applying production: A -> a A
  Applying production: A -> b
Tracing: [Symbols: a A A  | Input position: 0/3] Current symbol: a
  Matching terminal: a
Tracing: [Symbols: b A  | Input position: 0/3] Current symbol: b
Tracing: [Symbols: A A  | Input position: 1/3] Current symbol: A
  Applying production: A -> a A
  Applying production: A -> b
Tracing: [Symbols: a A A  | Input position: 1/3] Current symbol: a
Tracing: [Symbols: b A  | Input position: 1/3] Current symbol: b
  Matching terminal: b
Tracing: [Symbols: A  | Input position: 2/3] Current symbol: A
  Applying production: A -> a A
  Applying production: A -> b
Tracing: [Symbols: a A  | Input position: 2/3] Current symbol: a
Tracing: [Symbols: b  | Input position: 2/3] Current symbol: b
  Matching terminal: b
String is accepted.
student@administrator-VirtualBox:~/Desktop/22BLC1385_compiler$
```

Conclusion:

In this program, we have implemented a **bottom-up parser** using a **queue-based approach** to simulate the parsing of a string against a given **context-free grammar** (CFG). The key feature of this implementation is the ability to **trace the parsing process**, which is helpful in understanding how the input string is gradually processed according to the grammar's productions.

The **tracing** feature was added to ensure that the path taken by the parser is clearly visualized. By printing each step, including the **symbol being processed** and **productions applied**, users can observe how the program moves through the grammar and matches the input string. This tracing of the path adds a level of **clarity** and **transparency**, making the parsing process more understandable, especially for educational purposes or debugging.

The approach demonstrates a **queue-based simulation** of a parsing algorithm where non-terminals are progressively expanded into their respective productions, and terminals are matched to the input string. If a string is successfully parsed, the program outputs “String is accepted”; otherwise, it outputs “String is rejected.”

To ensure **uniqueness** in this implementation, **tracing the parsing path** at each significant step is included. This feature allows the program to provide an in-depth look at how the parser processes the input string, providing a clear, step-by-step visualization of the parsing process.

Overall, the implementation serves as a functional and informative tool for demonstrating the principles of **context-free grammar parsing**, with added traceability for improved understanding and debugging.